

WEEK 2 TASK

SYSTEM ARCHITECTURE DESIGN & MOCKUP

Project Title

Student Management System

A high-level architecture design document describing the system overview, components, data flow, module interactions, technology choices, scalability, security, maintainability, external integration, and user-interface mockups.

Prepared for	Junior Software Developer Internship
Task	Week 2: System Architecture Design & Mockup
Project Type	Web-based Academic Information Management System

1. System Overview

The Student Management System (SMS) is a web-based software application designed to centralize academic and administrative information related to students. Educational institutions often manage student details, course enrollment, attendance, and user access through multiple records or manual processes. The proposed system brings these activities into one structured platform.

The main users are **Administrators**, **Teachers**, and **Students**. Administrators manage student profiles, courses, users, and overall system configuration. Teachers can view assigned students and record attendance. Students can securely view their personal profile, enrolled courses, and attendance information.

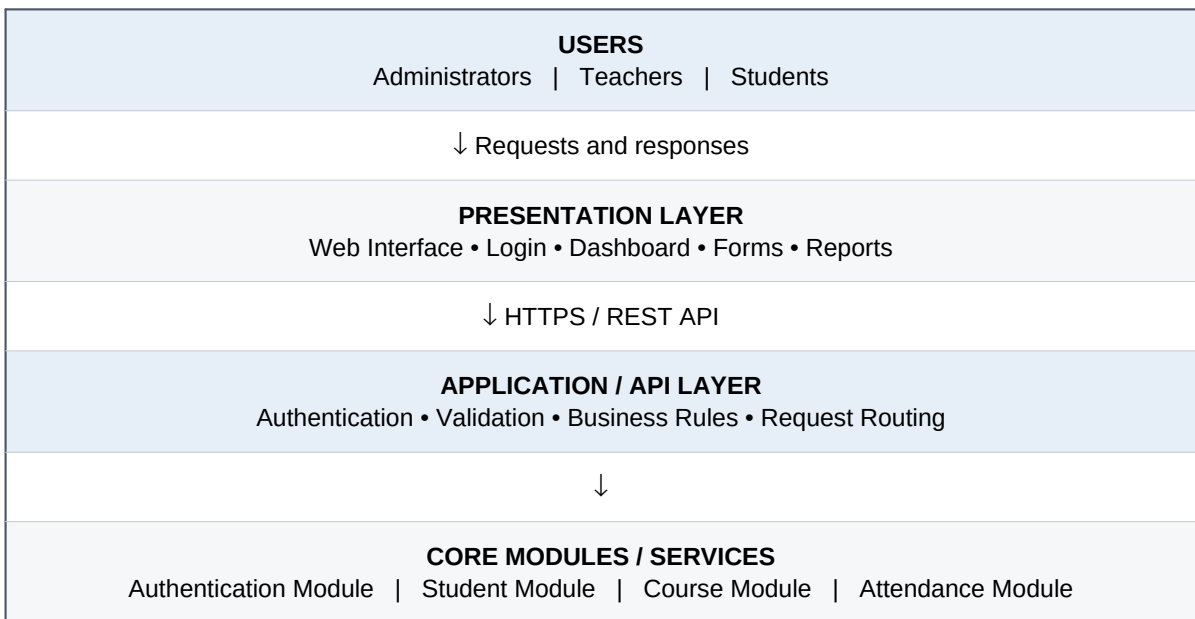
The architecture follows a **layered and modular design**. The presentation layer provides the user interface, the API/application layer receives and processes requests, independent service modules implement business functions, and the database layer stores persistent information. This structure reduces unnecessary dependencies and provides a strong foundation for future development.

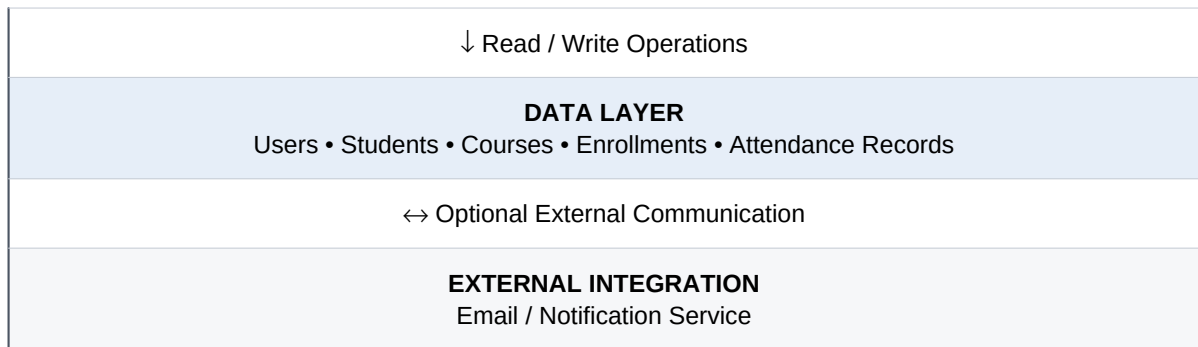
2. Architectural Design Approach

A layered architecture is selected because the project has clearly separated responsibilities. User-interface code should not directly manipulate database records, and database logic should not be mixed with presentation logic. Instead, requests move through controlled layers. This improves clarity, testing, security, and maintainability.

Layer	Responsibility
Presentation Layer	Provides pages, forms, dashboards, and user interaction.
Application / API Layer	Receives requests, validates data, applies business rules, and returns responses.
Service / Module Layer	Implements authentication, student, course, and attendance operations.
Data Layer	Stores and retrieves persistent information using a relational database.

3. High-Level System Architecture Diagram





The diagram shows the direction of communication. Users interact only with the presentation layer. The presentation layer sends structured requests to the API layer, which forwards validated operations to the relevant module. Modules access the database through controlled data operations and return the result through the same path.

4. Module and Component Description

4.1 Authentication and Authorization Module

Responsible for user login, logout, identity verification, password protection, and role-based access control. The module ensures that an Administrator, Teacher, or Student receives access only to permitted functions. It is used by all protected parts of the system and acts as a security boundary before sensitive operations are performed.

4.2 Student Management Module

Manages the student lifecycle within the application. Authorized users can create, view, search, update, and maintain student profiles. Typical data includes student ID, name, contact information, department, and enrollment-related details. Validation rules prevent incomplete or incorrectly formatted information from being stored.

4.3 Course Management Module

Maintains courses and student-course relationships. It supports creating course information, updating details, and associating students with appropriate courses. The module depends on valid student and course records and communicates with the database to maintain enrollment information.

4.4 Attendance Module

Allows teachers or authorized staff to record attendance and enables students or administrators to view stored records. Attendance operations depend on valid student and course information. This module therefore communicates with the Student and Course modules through the application layer.

4.5 Database and Data Access Component

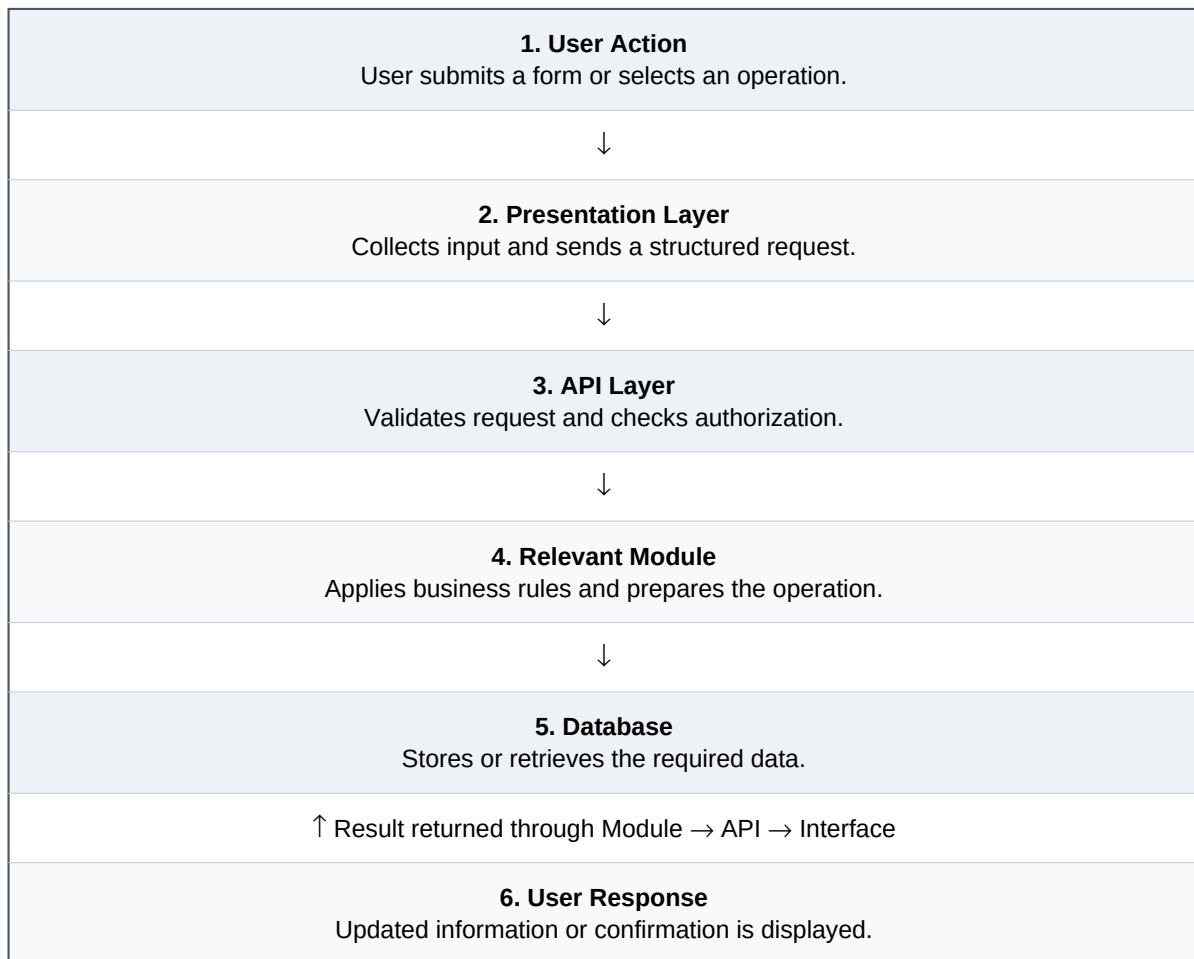
Stores persistent system information. The main logical entities are Users, Students, Courses, Enrollments, and Attendance Records. Relationships and constraints should protect data consistency, while database access should be performed through controlled queries or a data-access layer.

5. Component Interaction and Dependencies

Component	Depends On	Interaction
User Interface	API Layer	Sends user actions and displays responses.
Authentication	Users Data	Validates identity and user role.
Student Module	Database	Creates and manages student records.
Course Module	Students + Database	Maintains courses and enrollments.
Attendance Module	Students + Courses + Database	Records and retrieves attendance.
Notification Integration	Application Events	Sends optional email or alerts.

6. Data Flow Diagram

The following flow demonstrates how a typical request is processed. The same pattern can be used for student registration, course enrollment, attendance submission, and profile updates.



7. Data Flow Example: Marking Attendance

A teacher first logs in through the user interface. The Authentication Module verifies the teacher's credentials and role. The teacher selects a course and submits attendance data. The API validates the request and forwards it to the Attendance Module. The Attendance Module confirms that the selected course and students exist, then writes the attendance records to the database. After successful storage, a confirmation response is returned to the teacher's dashboard. If notification features are enabled, an application event may also be sent to an external email or notification service.

8. External Integration

External integration is optional but useful for a real-world version of the system. An email or notification service can receive events from the application layer, such as successful registration, attendance alerts, or important announcements. The integration should communicate through a clearly defined API so that the external provider can be replaced without redesigning the core modules.

9. Technology Stack and Justification

Area	Suggested Technology	Reason
Frontend	HTML, CSS, JavaScript / React	Creates responsive pages and reusable UI components.
Backend	Python / Java / Node.js	Supports business logic and REST API development.
Database	MySQL or PostgreSQL	Suitable for structured student and attendance data.
API	REST with JSON	Provides a simple communication contract between UI and backend.
Authentication	Secure hashing + sessions/tokens	Protects user accounts and supports controlled access.

10. Scalability, Security, and Maintainability

Scalability: The modular design allows the system to grow without redesigning every component. If the number of users increases, the application layer can be deployed on additional servers or instances. Database performance can be improved through indexing, optimized queries, backups, and future scaling strategies. Separating modules also makes it easier to improve high-demand features independently.

Security: Security begins with authentication and role-based authorization. Passwords should never be stored as plain text; secure password hashing must be used. The application should validate input, restrict database access, protect sessions or tokens, and use HTTPS in deployment. Administrators should have privileged operations, while students should only access their own permitted information.

Maintainability: Clear separation of responsibilities makes future changes safer. Each module has a defined purpose, and communication occurs through controlled interfaces. Developers can test modules independently, update one feature with reduced impact on unrelated components, and document APIs and database structures for easier maintenance.

11. User Interface Mockups

The following low-fidelity mockups represent the proposed interface structure. They are included to demonstrate how users may interact with the architecture described above.

11.1 Login Screen

STUDENT MANAGEMENT SYSTEM
Username / Email: _____
Password: _____
[LOGIN]
Forgot Password? Create Account

11.2 Administrator Dashboard

STUDENT MANAGEMENT SYSTEM
Dashboard Students Courses Attendance Reports Profile
DASHBOARD SUMMARY
Total Students Total Courses Attendance Overview
QUICK ACTIONS
+ Add Student Manage Courses Mark Attendance View Reports

11.3 Student Management Screen

STUDENT MANAGEMENT
[Search Student _____] [+ Add Student]

Student ID	Name	Department	Status	Actions
ST001	Student Name	CSE	Active	View / Edit
ST002	Student Name	CSE	Active	View / Edit

12. Conclusion

The proposed Student Management System uses a layered, modular architecture to provide a clear separation between user interaction, application processing, business functionality, and data storage. The main modules—Authentication, Student Management, Course Management, and Attendance—have defined responsibilities and interact through the application layer. The architecture supports controlled data flow, future external integration, and easier maintenance.

The selected design is appropriate because it balances simplicity for an initial implementation with the ability to grow. The architecture can support additional features such as reports, notifications, analytics, or mobile access in later development stages. Security, scalability, and maintainability have been considered from the design stage, making this architecture a suitable foundation for implementing the Student Management System.